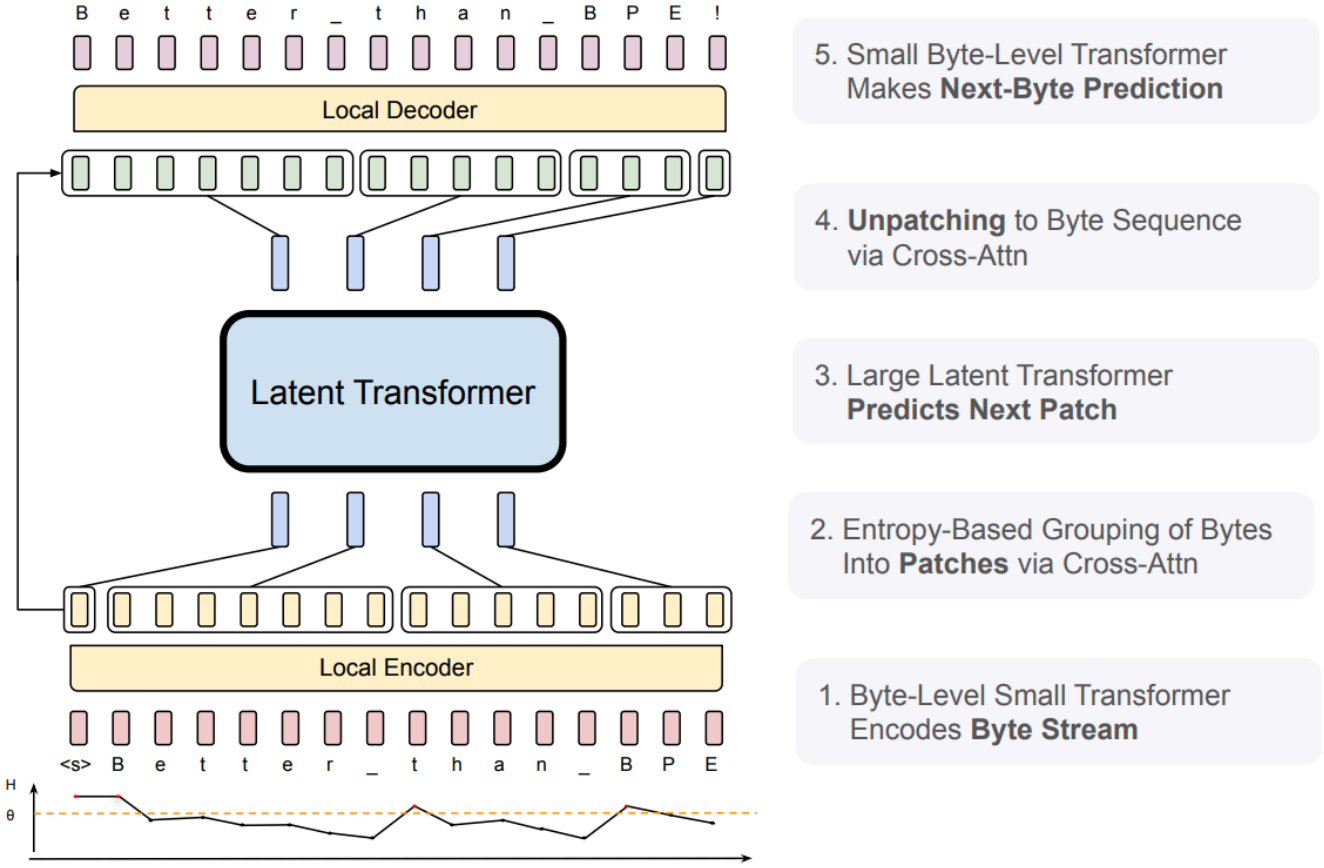
Byte Latent Transformer: Patches Scale Better Than Tokens

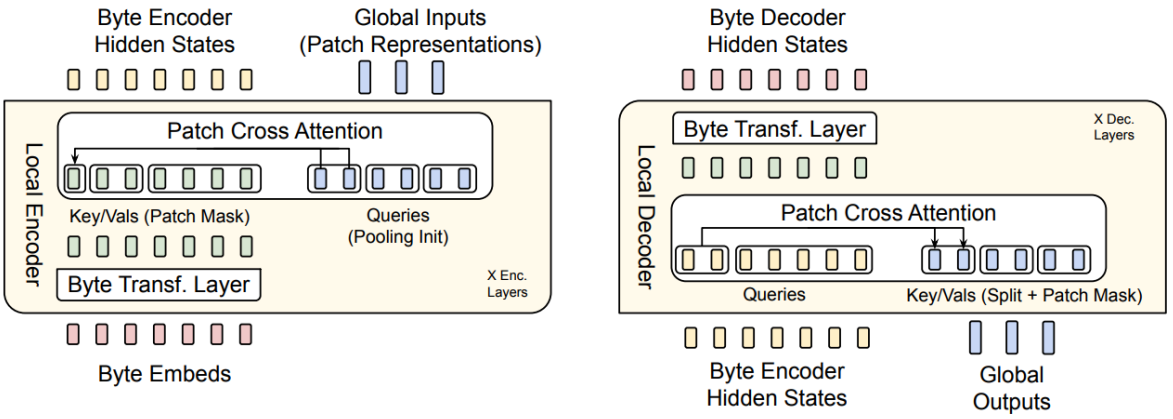
> #Architecture



- What problems does this project solve?
  - The current LLMs heavily rely on tokenizers which turn the input into the tokens in the corpus to then obtain the embeddings.
  - This method is affected by the spelling, noise, inter-language, and multimodal information significantly.
  - One classic way to solve this is via byte-level embedding. However, for a very long sequence of input, this can cause computational overhead.
- How does this architecture work?
  - **Entropy Patching**: When the next byte to be predicted has relatively high entropy (less confident, more uncertainty), often meaning the start of a new word or entity, then a cut will be added here. If the next byte has low entropy, meaning it is possible to be in the middle of a word or entity, then this byte will be included in a patch.

    $H(x_i) = \sum_{v \in \mathcal{V}} p_e(x_i = v|x_{<i}) \log\ p_e(x_i = v|x_{<i}) \leq \theta_g$, where θ_g is a global threshold.

    $H(x_t) - H(x_{t-1}) > \theta_r$, meaning we want to find a point that the entropy increases significantly to end/start a patch.
  - **Architecture**:



- Local Encoder: Group the bytes and turn them into patch-based representations
  - Embed each byte.
  - Set up an n-gram hash table, fetch the n-gram and map it to an embedding from the table. Add the n-gram embedding to the byte embedding.
  - Local Block Causal Attention: Given a window, run causal attention in the window bytewise-ly.
  - Cross Attention Pooling: Use patch as Q, and the bytes as the K, V to run cross attention.
- Latent Global Transformer: Where the main work happens. This part works and generates autogressively in latent space. Here, the causal attention is calculated among the patches Q, K, V.
- Local Decoder: Translate from latent space back to bytes.
  - Cross attention: Use bytes as Q, and patches as K, V.
  - Byte-level Transformer Layer (byte-> byte)

- Experiment Results
  - Performance and Efficiency Balance: At a parameter scale of up to 8B, BLT demonstrates for the first time that a tokenizer-free byte-level model can achieve performance comparable to Llama 3 (using a BPE tokenizer). Simultaneously, it reduces computational effort (FLOPs) by up to 50% during inference.
  - Superior Scaling Laws: Within a fixed inference computational budget, BLT enables researchers to simultaneously increase both “model parameter count” and “average patch size.” Larger patches conserve computational resources, allowing the model core to scale larger—resulting in better scaling trends than traditional LLMs.
  - Exceptional Robustness: Direct byte-level modeling grants BLT far superior resilience to noisy inputs (e.g., mixed case, random character deletions) compared to Llama 3 or even Llama 3.1. It excels in spelling correction, phonological processing, and low-resource language translation.
- Work Cited
  - Paper
  - NotebookLM
  - Chat History